

Trabajo Practico: Drivers, Conexión y Modelado con ODM

Ignacio Figueroa – Comisión 7

Base de Datos II

Parte 1: Implementación con Driver Nativo

Archivo: driver_lab.js

```
import { MongoClient } from 'mongodb';

const url = 'mongodb://localhost:27017';
const client = new MongoClient(url);
const dbName = 'laboratorio';

async function main() {
  try {
    await client.connect();
    console.log('Conectado exitosamente al servidor de MongoDB');

    const db = client.db(dbName);
    const collection = db.collection('componentes');

    const sensoresRobotica = [
      { nombre: 'Sensor de Humedad', tipo: 'Sensor', estado: 'Activo' },
      { nombre: 'Servo Motor', tipo: 'Actuador', estado: 'Activo' },
      { nombre: 'LED RGB', tipo: 'Salida', estado: 'Inactivo' }
    ];

    const insertResult = await collection.insertMany(sensoresRobotica);
    console.log(`Se insertaron ${insertResult.insertedCount} componentes de robótica correctamente.`);

    } catch (error) {
      console.error('Error durante la ejecución:', error);
    } finally {
      await client.close();
    }
  }

  main();
}
```

Parte 2: Implementación con Mongoose (ODM)

Archivo: mongoose_lab.js

```
import mongoose from 'mongoose';

const url = 'mongodb://localhost:27017/laboratorio';

async function main() {
  try {
    await mongoose.connect(url);
    console.log('Conectado a MongoDB mediante Mongoose');

    // Definición del Esquema con validaciones y timestamps
    const componenteSchema = new mongoose.Schema({
      nombre: {
        type: String,
        required: true
      },
      stock: {
        type: Number,
        default: 0
      }
    }, {
      timestamps: true
    });

    // Creación del Modelo
    const Componente = mongoose.model('Componente', componenteSchema);

    // Inserción de prueba
    const nuevoComponente = new Componente({
      nombre: 'Sensor Ultrasonico HC-SR04',
      stock: 15
    });

    const resultado = await nuevoComponente.save();
    console.log('Inserción de prueba exitosa utilizando Mongoose:', resultado);

  } catch (error) {
    console.error('Error en la implementación de Mongoose:', error);
  } finally {
    await mongoose.disconnect();
  }
}

main();
```

Parte 3: Reflexión Final

1. ¿Cuál es la función del driver como "traductor" entre la aplicación y MongoDB?

El driver actúa como el puente de comunicación directo de bajo nivel. Su función es traducir los comandos, métodos y estructuras que escribimos en nuestro lenguaje de programación (en este caso, JavaScript/Node.js) al protocolo de cable y comandos internos que el motor de MongoDB entiende de forma nativa, permitiendo que la aplicación mande consultas, inserte datos y reciba las respuestas del servidor de la base de datos sin problemas de compatibilidad.

2. ¿A qué formato convierte el driver las estructuras de datos nativas (como objetos de JavaScript) para enviarlas al servidor?

El driver convierte los objetos nativos de JavaScript al formato **BSON** (Binary JSON). Aunque en el código manejamos objetos tradicionales o JSON, MongoDB almacena y procesa la información en un formato binario que es mucho más eficiente para el almacenamiento, la indexación y la velocidad de parseo en el servidor.

3. Según lo visto, ¿en qué situación de un proyecto de ingeniería elegirían usar el driver nativo en lugar de Mongoose?

Elegiría usar el driver nativo en escenarios de alta carga o sistemas críticos donde el rendimiento puro sea la prioridad absoluta. Por ejemplo:

- Procesos de ingesta masiva de datos en tiempo real (como procesamiento de logs o telemetría de dispositivos IoT que mandan ráfagas constantes de información).
- Scripts de migración de datos pesados donde la sobrecapa de abstracción, validación y hooks de ciclo de vida de Mongoose penalizaría la velocidad de ejecución de manera innecesaria.
- Microservicios hiper-especializados y minimalistas que solo requieran hacer lecturas o escrituras directas muy rápidas sin lógica de negocio compleja adjunta.